

CHAPTER III 建模方式

CHAPTER III 建模方式

数据流建模

行为级建模

过程语句

initial语句

always语句块

语句块

过程赋值语句

过程连续赋值语句

条件分支语句

循环语句

结构化建模

模块级建模:

1. 模块调用方式
2. 模块端口对应方式

门级建模

开关级建模:

数据流建模

描述所有组合电路! assign+运算符

3.1.1 连续赋值语句

— 连续赋值的目标类型主要是标量线网和向量线网两种。

(1) 标量线网, 如: wire a,b;

(2) 向量线网, 如: wire [3:0]a,b;

显式连续赋值语句:

- <net_declaration><range><name>;
- assign #<delay><name>= Assignment expression;

隐式连续赋值语句:

- <net_declaration><drive_strength><range>#<delay><name>= Assignment expression;

•<net_declaration>(连线型变量类型)

•<range>(变量位宽), 指明了变量数据类型的宽度, 格式为[msb:lab], 缺省为1位。

•<drive_strength>(赋值驱动强度)是可选的, 只能在“隐式连续赋值语句”格式中得到指定。它用来对连线型变量受到的驱动强度进行指定。

wire (weak0, strong1) out=in1&in2;

•<delay>(延时量)这一项是可选的。
// (delay1, delay2, delay3)

例3.1-1 显式连续赋值语句的例子

```
module example1_assignment(a,b,m,n,c,y);  
input[3:0]a,b,m,n;  
output[3:0]c,y;  
wire[3:0]a,b,m,n,c,y;  
assign y=m|n;  
assign #(3, 2, 4)c=a&b;  
endmodule
```

例3.1-2 隐式连续赋值语句的例子

```
module example2_assignment(a,b,m,n,c,y,w);  
input[3:0]a,b,m,n;  
output[3:0]c,y,w;  
wire[3:0]a,b,m,n;  
wire[3:0]y=m|n;  
wire[3:0]//(3,2,4)c=a&b;  
wire(strong0,weak1)[3:0]#(2,1,3)w=(a^b)&(m^n);  
endmodule
```

连续赋值语句需要注意的以下几点:

- 赋值目标只能是线网类型 (wire);
- 在连续赋值中, 只要赋值语句右边表达式任何一个变量有变化, 表达式立即被计算, 计算的结果立即赋给左边信号 (若没有定义延时量);
- 连续赋值语句不能出现在过程块中。
- 多个连续赋值语句之间是并行语句, 因此与位置顺序无关。
- 连续赋值语句中的延时具有硬件电路中惯性延时的特性, 任何小于其延时的信号变化脉冲都将被滤除掉, 不会体现在输出端口上。

并行化设计

行为级建模

类别	语句	可综合性
过程语句	initial	
	always	√
语句块	串行语句块begin-end	√
	并行语句块fork-join	
赋值语句	过程连续赋值assign	
	过程赋值=、<=	√
条件语句	if-else	√
	case, case02, caseX	√
循环语句	forever..	
	repeat	

过程语句

initial语句

1. initial过程语句

initial过程语句的语法格式为：

```

initial
begin
    语句1;
    语句2;
    . . .
    语句n;
end

```

例3.2-1 用initial过程语句对变量A,B,C进行赋值

```

module initial_tb1;
reg A,B,C;
initial
begin
    A=0;B=1;C=0;
    #100 A=1;B=0;
    #100 A=0;C=1;
    #100 B=1;
    #100 B=0;C=0;
end
endmodule

```

initial语句主要用来产生一些测试量

always语句块

always语句块

从语法描述角度，相对于initial过程块，always语句块的触发状态是一直存在的，只要满足always后面的敏感事件列表，就执行过程块。

其语法格式是：

```
always@(<敏感事件列表>)  
    语句块;
```

例如：^

@(a)	//当信号a的值发生改变时
@(a or b)	//当信号a或信号b的值发生改变时
@(posedge clock)	//当clock的上升沿到来时
@(negedge clock)	//当clock的下降沿到来时
@(posedge clk or negedge reset)	//当clk的上升沿到来或reset信号的下降沿到来时

在信号定义形式方面，无论是对时序逻辑还是组合逻辑描述，Verilog HDL要求在过程语句（initial和always）中，被赋值信号必须定义为“reg”类型。

在敏感事件表方面，这是Verilog HDL语言中一个关键性设计，如何选取敏感事件作为过程的触发条件，在Verilog HDL程序中有一定的设计要求。

（1）采用过程对组合电路进行描述时，作为全部的输入信号需要列入敏感信号列表。

（2）采用过程对时序电路进行描述时，需要把时间信号和部分输入信号列入敏感信号列表。应当注意的是，不同的敏感事件列表会产生不同的电路形式。

在信号定义形式方面，无论是对时序逻辑还是组合逻辑描述，Verilog HDL要求在过程语句（initial和always）中，被赋值信号必须定义为“reg”类型。

在敏感事件表方面，这是Verilog HDL语言中一个关键性设计，如何选取敏感事件作为过程的触发条件，在Verilog HDL程序中有一定的设计要求。

（1）采用过程对组合电路进行描述时，作为全部的输入信号需要列入敏感信号列表。

（2）采用过程对时序电路进行描述时，需要把时间信号和部分输入信号列入敏感信号列表。应当注意的是，不同的敏感事件列表会产生不同的电路形式。

四选一数据选择器的编写:

例3.2-2 用initial语句产生测试信号

```
module initial_tb2;
reg S1; //被赋值信号定义“reg”类型
initial
begin
    S1=0;
    #100    S1=1;
    #200    S1=0;
    #50     S1=1;
    #100    $finish;
end
endmodule
```

例3.2-3 用always语句描述4选1数据选择器
4选1数据选择器是一种典型的组合逻辑电路，其Verilog HDL程序代码是：

```
module mux4_1(out,in0,in1,in2,in3,sel);
output out;
input in0,in1,in2,in3;
input[1:0] sel;
reg out; //被赋值信号定义为“reg”类型
always @( in0 or in1 or in2 or in3 or sel)
//敏感信号列表
    case(sel)
        2'b00:    out=in0;
        2'b01:    out=in1;
        2'b10:    out=in2;
        2'b11:    out=in3;
        default:   out=2'bx;
    endcase
endmodule
```

例3.2-4用always语句描述同步置数、同步清零计数器

```
module counter1 (out,data,load,reset,clk);
output[7:0] out;
input[7:0] data;
input load,clk,reset;
reg[7:0] out;

always @(posedge clk) //clk上升沿触发
begin
    if (!reset) out = 8'h00;
        //同步清0，低电平有效
    else if (load) out = data;
        //同步置数
    else out = out + 1;
end
endmodule
```

例3.2-5 用always过程语句描述异步清零计数器

```
module counter2(clear,clk,out);
output[7:0] out;
input clk,clear;
reg[7:0] out;

always @(posedge clk or negedge clear)
//clk上升沿和clear低电平清零有效
begin
    if(!clear) //异步清零
        out=0;
    else out=out+1;
end
endmodule
```

同步复位信号的兴起与异步复位信号的消失

语句块

语句块包括串行语句块（**begin-end**）和并行语句块（**fork-join**）两种。

串行语句块采用关键字“**begin**”和“**end**”，其中的语句按串行方式顺序执行，可以用于可综合电路程序和仿真测试程序。其语法格式是：

begin :块名	fork :块名
块内声明语句;	块内声明语句;
语句1;	语句1;
语句2;	语句2;
...	...
语句n;	语句n;
end	join

并行语句执行：

（1）采用串行语句块的Verilog HDL仿真程序代码是：

```
module wave_tb1;
reg wave;
parameter T=10;
initial
begin
    wave=0;
    #T    wave=1;
    #T    wave=0;
    #T    wave=1;
    #T    wave=0;
    #T    wave=1;
end
endmodule
```

（2）采用并行语句块的Verilog HDL仿真程序代码是：

```
module wave_tb2;
reg wave;
parameter T=10;
initial
fork
    wave=0;
    #T    wave=1;
    #(2*T) wave=0;
    #(3*T) wave=1;
    #(4*T) wave=0;
    #(5*T) wave=1;
join
endmodule
```

注意到，**fork join**语句是并行执行的，其延迟是绝对延迟，而串行的**begin end**则是相对延迟。

语句块	串行语句块 (begin-end)	并行语句块 (fork-join)
执行顺序	按照语句顺序执行	每条语句在同一时刻执行
语句前面延时的意义	相对于前一条语句执行结束的相对时间	相对于并行语句块启动时的时间
起始时间	首句开始执行的时间	转入并行语句块的时间
结束时间	最后一句语句执行结束时的时间	执行时间最长的那条语句执行结束时的时间
行为描述的意义	电路中数据在时钟及控制信号作用下, 沿数据通道的各级寄存器之间的传递过程	电路上电后, 各电路模块同时开始工作的过程

过程赋值语句

阻塞性赋值语句 **=** "先后"

阻塞赋值语句有如下特点:

- (1) 在串行语句块中, 各条阻塞赋值语句将按照先后排列顺序依次执行; 在并行语句块中的各条阻塞赋值语句则同时执行, 没有先后顺序之分;
- (2) 执行阻塞赋值语句的顺序是, 先计算等号右端表达式的值, 然后立刻将计算的值赋给左边的变量, 与仿真时间无关。

非阻塞性赋值语句 **<=** "同时"

非阻塞赋值语句有如下特点:

- (1) 在串行语句块中, 各条非阻塞赋值语句的执行没有先后顺序之分, 排在前面的语句不会影响到后面的语句的执行, 各条语句并行执行;
- (2) 执行非阻塞赋值语句的顺序是, 先计算右端表达式的值, 然后等到延时时间结束时, 将计算的值赋给左边的变量。

体现它们的不同, 注意非阻塞性是并行执行的:

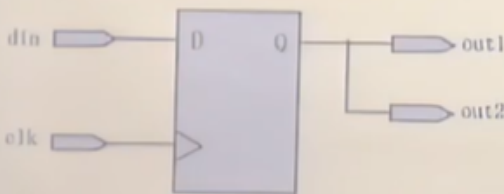
例3.2-7: 试分析下面两段Verilog HDL程序所描述的电路结构

程序 (1)

```
module block1(din,clk,out1,out2);
input din,clk;
output out1,out2;
reg out1,out2;
always@(posedge clk)
begin
```

```
out1=din;
out2=out1;
```

```
end
endmodule
```

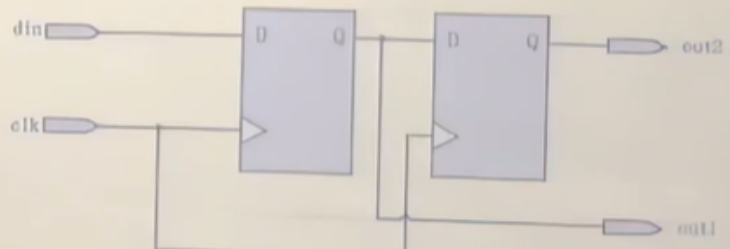


程序 (2)

```
module non_block1 (din,clk,out1,out2);
input din,clk;
output out1,out2;
reg out1,out2;
always@(posedge clk)
begin
```

```
out1<=din;
out2<=out1;
```

```
end
endmodule
```



前者为阻塞性质,所以din要先赋给out1,然后out1再赋给out2

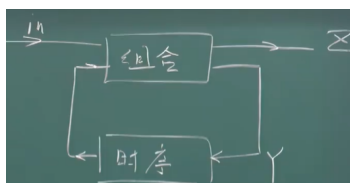
后者为非阻塞性质,因而与先后顺序无关,那么如果使用阻塞性语句来实现两个触发器的综合,则将它们顺序颠倒即可:

在这两个程序中,基本描述相同,不同的是程序(1)采用了阻塞赋值语句,而程序(2)采用了非阻塞赋值语句。

如果采用阻塞赋值语句描述程序(2)的电路,其Verilog HDL 程序代码是:

```
module block2(din,clk,out1,out2);
input din,clk;
output out1,out2;
reg out1,out2;
always@(posedge clk)
begin
out2=out1;
out1=din;
end
endmodule
```

流水线的设计概念



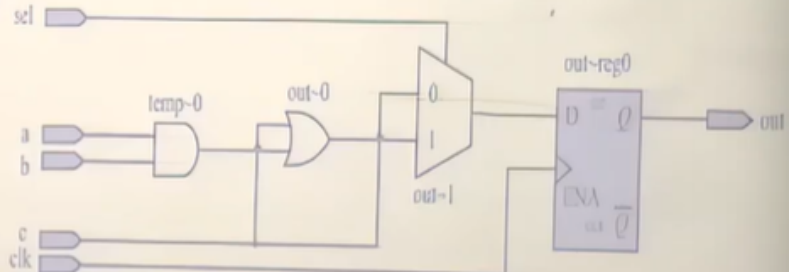
速度提高的方法

输出方程,状态方程和状态转移方程

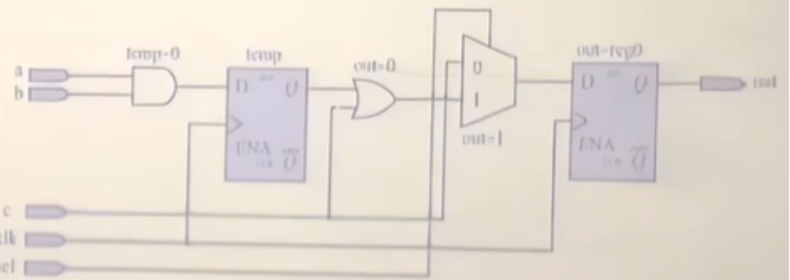
例3.2-8: 试分析下面两段Verilog HDL程序所描述的电路结构

程序 (1)

```
module block3(a,b,c,clk,sel,out)
input a,b,c,clk,sel;
output out;
reg out,temp;
always@(posedge clk)
begin
    temp=a&b;
    if(sel) out=temp|c;
    else    out=c;
end
endmodule
```



```
module non_block2(a,b,c,clk,sel,out);
input a,b,c,clk,sel;
output out;
reg out,temp;
always@(posedge clk)
begin
    temp<=a&b;
    if(sel) out<=temp|c;
    else    out<=c;
end
endmodule
```



只有在行为级建模的串行语句块中间使用阻塞型语句才是串行结构

过程连续赋值语句

在Verilog HDL中, 过程性连续赋值语句有两种类型: 赋值、重新赋值语句 (assign、deassign) 和强制、释放语句 (force、release)。

赋值语句和重新赋值语句采用的关键字是"assign"和"deassign", 语法格式分别是:

assign <寄存器型变量> = <赋值表达式>;

deassign <寄存器型变量>;

不可综合语句.

3.2-9 使用assign和deassign设计异步清零D触发器

```
module assign_dff(d,clr,clk,q);
input d,clr,clk;
output q;
reg q;
always@(clr)
begin
    if(!clr)
        assign q=0;           //时钟沿来临时，d的变化对q无效。
    else
        deassign q;
end

always@(negedge clk) q=d;
endmodule
```

强制语句和释放语句采用的关键字是“force”和“release”，可以对连线型和寄存器型变量类型进行赋值操作，“force”语句的优先级高于“assign”语句。语法格式分别是：

force <寄存器或连线型变量> = <赋值表达式>;

release<寄存器或连线型变量>;

例3.2-10 force和release使用例程

```
module force_release(a,b,out);
input a,b;
output out;
wire out;
and #1(out,a,b);
initial
begin
    force out=a|b;
    #5;
    release out;
end
endmodule
```

```
module release_tb;
reg a,b;
wire out;
force_release U1(a,b,out);
initial
begin
    a=1;b=0;
end
endmodule
```

条件分支语句

例3.2-11 if-else使用例程

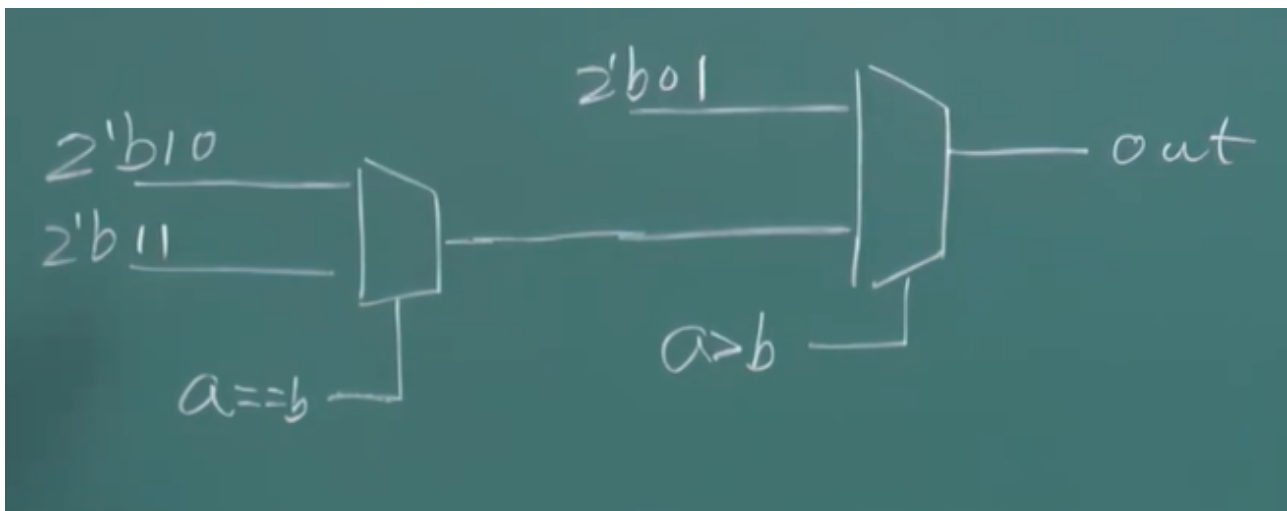
```
module mux2_1(a,b,sel,out);
input a,b,sel;
output out;
reg out;
always@(a,b,sel)
begin
if(sel)    out=a;
else      out=b;
end
endmodule
```

当sel为真“1”时，输出端out得到a的值，而当sel为假“0”时，输出端out得到b的值。这是一个典型的二选一的数据选择器。

例3.2-11 if-else使用例程

```
module compare_a_b(a,b,out);
input a,b;
output [1:0]out;
reg [1:0]out;
always@(a,b)
begin
if(a>b)    out=2'b01;
else if(a==b) out=2'b10;
else      out=2'b11;
end
endmodule
```

例中，首先判断a是否大于b，然后判断a是否等于b，蕴含了优先级的特性，这种特性会在综合后的电路中体现出来。



对于多个判断:使用switch case语句

case	0	1	x	z
0	1	0	0	0
1	0	1	0	0
x	0	0	1	0
z	0	0	0	1

使用case语句需要注意的是：

(1) 值1到值n之间必须各不相同，一旦判断到与某值相同并执行相应语句块后，case语句的执行便结束。

(2) 如果某几个连续排列的值项执行的是同一条语句，则这几个值项间可用逗号相隔，而将语句放在这几个值项的最后一个中。

(3) default选项相当于if-else语句中的else部分，可依据需要用或者不用，当前面已经列出了敏感表达式的所有可能值，则default可以省略。

(4) case语句的所有表达式的值的位宽必须相等，只有这样控制表达式和分支表达式才能进行对应位的比较。

除了case分支语句以外，还有casez、casex这两种功能类似的条件分支语句

casez	0	1	x	z
0	1	0	0	1
1	0	1	0	1
x	0	0	1	1
z	1	1	1	1

casex	0	1	x	z
0	1	0	1	1
1	0	1	1	1
x	1	1	1	1
z	1	1	1	1

例题:数码管显示:

例3.2-12: 用case语句描述的BCD数码管译码

```
module BCD_decoder(out,in);
output[6:0]out;
input[3:0]in;
reg [6:0]out;
always@(in)
begin
    case(in)
        4'd0:out=7'b1111110;
        4'd1:out=7'b0110000;
        4'd2:out=7'b1101101;
        4'd3:out=7'b1111001;
        4'd4:out=7'b0110011;
        4'd5:out=7'b1011011;
        4'd6:out=7'b1011111;
        4'd7:out=7'b1110000;
        4'd8:out=7'b1111111;
        4'd9:out=7'b1111011;
        default:out=7'bx;
    endcase
end
endmodule
```

字形	输入	输出
	in	ABCDEFG
0	0000	1111110
1	0001	0110000
2	0010	1101101
3	0011	1111001
4	0100	0110011
5	0101	1011011
6	0110	1011111
7	0111	1110000
8	1000	1111111
9	1001	1111011

在使用case语句时，应包含所有状态，如果没包含全，那么缺省项必须写，否则将产生锁存器，这在同步时序电路设计中是不允许的。

例3.2-13: case语句的使用例程
程序（1）会产生锁存器的case语句

```
module latch_case(a,b,sel,out);
input a,b;
input [1:0]sel;
output out;
reg out;
always@(a,b,sel)
    case(sel)
        2'b00:out=a;
        2'b11:out=b;
    endcase
endmodule
```

程序（2）不会产生锁存器的case语句

```
module non_latch_case(a,b,sel,out);
input a,b;
input [1:0]sel;
output out;
reg out;
always@(a,b,sel)
    case(sel)
        2'b00:out=a;
        2'b11:out=b;
        default:out=0;
    endcase
endmodule
```

对于左图,输出端直接与输入端相连,构成了锁存器

循环语句

1. forever循环语句

关键字“forever”所引导的循环语句表示永久循环。在永久循环中不包含任何条件表达式，只执行无限的循环，直到遇到系统任务\$finish为止。如果需要从forever循环中退出，则可以使用disable语句。forever语句的语法格式是：

forever 语句或语句块；

例3.2-14：用 forever语句产生时钟信号

```
module forever_tb;
reg clock;
initial
begin
    clock=0;
    forever #50      clock=~clock;
end
endmodule
```

Repeat 语句引导的循环语句表示执行固定次数的循环：

格式是：

repeat (循环次数表达式)
语句或语句块 (循环体) ;

例3.2-15：使用repeat循环语句产生固定周期数时钟信号

```
module repeat_tb;
reg clock;
initial
begin
    clock=0;
    repeat(8) #50 clock=~clock;
end
endmodule
```

3. while循环语句

关键字“while”所引导的循环语句表示的是一种“条件循环”。while语句根据条件表达式的真假来确定循环体的执行，当指定的条件表达式取值为真时才会重复执行循环体，否则就不执行循环体，其语法格式是：

while(条件表达式) 语句或语句块；

例3.2-16：使用while语句产生时钟信号

```
module while_tb;
reg clock;
Initial
begin
    clock=0;
    while(1)
    #50      clock=~clock;
end
endmodule
```

关键字“for”所要引导的循环语句也表示一种“条件循环”，只有在指定的条件表达式成立时才进行循环，其语法格式是：

for (循环变量赋初值；循环结束条件；循环变量增值) 语句块；

例3.2-17：使用for语句产生时钟信号

```
module for_clk;
reg clk;
integer i;
initial
begin
clk=0;
for(i=0;i<=10;i=i+1)
#50 clk=~clk;
end
endmodule
```

例3.2-18：用Verilog HDL语言设计一个8位移位寄存器

程序（1）采用赋值语句实现

```
module shift_regist1 (Q,D,rst,clk);
output [7:0] Q;
input D,rst,clk;
reg [7:0] Q;
always @(posedge clk)
if (!rst) Q<=8'b0000000;
else Q<={Q[6:0],D};
endmodule
```

程序（2）采用“for”循环语句实现

```
module shift_regist2 (Q,D,rst,clk);
output [7:0] Q;
input D,rst,clk;
reg [7:0] Q;
integer i;
always @(posedge clk)
if (!rst) Q<=8'b0000000;
else
begin
for (i=7;i>0;i=i-1) Q[i]<=Q[i-1];
Q[0]<=D;
end
endmodule
```

结构化建模

结构描述方式就是将硬件电路描述成一个分级子模块系统，通过逐层调用这些模块构成功能复杂的数字逻辑电路和系统的一种描述方式。在这种描述方式下，组成硬件电路的各个子模块之间的相互层次关系以及相互连接关系都需要得到说明。

根据所调用子模块的不同抽象级别，可以将模块的结构描述方式分成如下三类：

(1)模块级建模:通过调用由用户设计生成的低级子模块来对硬件电路结构进行说明，这种情况下模块由低级模块的实例组成。

(2)门级建模:通过调用Verilog HDL内部的基本门级元件来对硬件电路的结构进行说明，这种情况下模块将由基本门级元件的实例组成。

(3)开关级建模:通过调用Verilog HDL内部的基本开关元件来对硬件电路的结构进行说明，这种情况下模块将由基本开关级元件的实例组成。

模块级建模就是通过调用由用户自己描述产生的**module**模块对硬件电路结构进行说明，并设计出电路。

模块建模方式可以把一个模块看作是由其它模块像积木一样搭建而成。模块中被调用模块属于低一层次的模块，如果当前模块不再被其它模块所调用，那么这个模块一定是所设的顶层模块。在对个硬件系统的描述中，必定有而且只能有一个顶层模块。

模块级建模：

1. 模块调用方式

在Verilog HDL中，模块可以被任何其它模块调用，这种调用实际上是将模块所描述的电路复制并连接的。其中模块调用的基本语法格式是
模块名<参数值列表>实例名(端口名列表)；

```
module and_2(a,b,c);  
    //2输入与门模块  
    input a,b;  
    output c;  
    assign c=a&b;  
endmodule  
  
module logic(in_1, in2,q);  
    //顶层模块,  
    input in1,in2;  
    output q;  
    and_2 u1(in1,in2,q);  
    //模块的调用  
endmodule
```

如果同一个模块在当前模块中被调用几次，则需要用不同的实例名加以标识，但可在同一条模块调用语句中被定义，只要各自的实例名和端口名列表相互间用逗号隔开即可。基本语法格式是：

模块名

<参数值列表>实例名1(端口名列表1)，<参数值列表>实例名2(端口名列表2)，...<参数值列表>实例名n(端口名列表n)；

在上面的格式当中，模块名就是被调用的模块，参数值列表是可选项，实例名代表生成的模块实例，实例名必须各不相同，端口列表指明了模块实例与外部信号的连接。

例3.3-2使用阵列调用方式的模块实例语句来进行结构描述

```

module AND(andout,ina,inb);//基本的与门模块
input ina,inb,
output andout;
assign andout=ina&inb;
endmodule

module ex_arrey(out,a,b);//顶层模块,用来调用与门模块
input[15:0] a,b;
output [15:0]out;
wire [15:0]out;
AND AND_ARREY[15:0](out,a,b);
endmodule

//其中模块调用语句等价于如下几条语句:
AND AND_ARREY15(out[15],a[15],b[15]);...
AND AND_ARREY1(out[1],a[1],b[1]);
AND AND_ARREY0(out[0],a[0],b[0]);

```

2.模块端口对应方式

2. 模块端口对应方式

(1) 端口位置对应方式

端口位置对应方式是被调用的模块按照一定的顺序出现在端口连接表中的一种模块调用方式。其语法格式是：

模块名 <参数值列表> 实例名 (<信号名1>, <信号名2>, ..., <信号名n>);

例3.3-3: 采用模块结构搭建方式用1bit半加器构成1bit全加器

```

module halfadder(a,b,s,c);
//半加器模块
input a,b;
output c,s;
assign s=a^b;
assign c=a&b;
endmodule

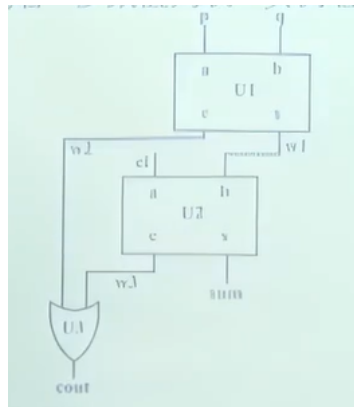
```

```

module fulladder(p,q,ci,co,sum); //全加器模块
input p,q,ci;
output co,su;
wire w1,w2,w3;
halfadder U1(p,q,w1,w2);
halfadder U2(ci,w1,sum,w3);
or U3(co,w2,w3);
endmodule

```

将上述的半加器加以组合构成全加器



(2) 端口名对应方式

端口名对应方式是Verilog HDL允许的另一种模块调用方式，其语法格式如下：

模块名 <参数值列表> 实例名 (.端口名1<信号名1>, .端口名2<信号名2>, ..., .端口名n<信号名n>);

例3.3-4 端口名对应的模块调用的例子

```

module dff(d,clk,clr,q); //D触发器模块，是被调用的模块，属于底层模块
input d,clk,clr;
output q;
reg q;
always@(posedge clk or negedge clr)
begin
    if(!clr)
        q=0;
    else
        q=d;
end
endmodule

module shifter_D(din,clock,clear,out); //顶层模块，用来调用底层模块
input din,clock,clear;
output [3:0]out;
dff U1(.q(out[0]),.d(din),.clk(clock),.clr(clear));
dff U2(.q(out[1]),.d(out[0]),.clk(clock),.clr(clear));
dff U3(.q(out[2]),.d(out[1]),.clk(clock),.clr(clear));
dff U4(.q(out[3]),.d(out[2]),.clk(clock),.clr(clear));
endmodule
  
```

(3) 不同端口位宽的匹配

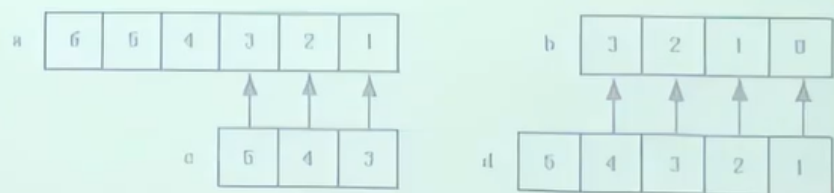
在端口和端口表达式之间存在一种隐含的连续赋值的语句。因此当端口和端口表达式的位宽不一致时，会进行端口的匹配，采用的位宽匹配规则与连续赋值时使用的规则相同。

例3.3-5 模块调用时不同位宽的匹配问题

```

module ex1(a,b);
input [6:1]a;
output [3:0]b;
...
endmodule

module test;
wire[5:3]c;
wire[5:1]d;
ex1 U1(.a(c),.b(d));
...
endmodule
  
```



其连接对应关系如图所示。

右对齐的方式。

4. 模块参数值

用下面两种途径可以改变模块实例的参数值，分别是使用带有参数的模块实例语句修改参数值和使用定义参数语句（defparam语句）修改参数值。

（1）使用带有参数的模块实例语句修改参数值

在这种方法中，模块实例的本身就能指定新的参数值，其语法格式是：

模块名 <参数值列表>调用名（端口名列表）；

其中参数值列表又分为位置对应和名称对应两种方式。

例3.3-6 模块调用改变参数值的例子

```
module para1(C,D);  
    parameter a=1;  
    parameter b=1;  
    . . .  
endmodule  
module para2;  
    . . .  
    para1 #(4,3) U1(C1,D1);  
    //语句1  
    para1 #(.b(6),.a(5)) U2(C2,D2);  
    //语句2  
    . . .  
endmodule
```

（2）使用定义参数语句（defparam语句）修改参数值

在进行模块调用时更改被调用模块内参数值的第二种方法就是利用“参数重定义语句”（defparam语句）。

参数重定义语句的语法格式是：

```
defparam      参数名1=参数值1,  
              参数名2=参数值2,  
              . . .  
              参数名n=参数值n;
```

需要注意的是，参数名必须采用分级路径的形式，才能锁定需要修改的参数是哪个模块当中的。

例3.3-7 使用defparam语句修改参数值

```
module halfadder(a,b,s,c);    //半加器模块  
halfadder  
    input a,b;  
    output c,s;  
    parameter xor_delay=2,and_delay=3;  
    assign #xor_delay s=a^b;  
    assign #and_delay c=a&b;  
endmodule  
  
module fulladder(p,q,ci,co,sum);    //全加器模块  
fulladder  
    input p,q,ci;  
    output co,sum;  
    parameter or_delay=1  
    wire w1,w2,w3;  
    halfadder    U1(p,q,w1,w2);  
    halfadder    U2(ci,w1,sum,w3);  
    or #or_delay U3(co,w2,w3);  
endmodule
```

```
module top1(top1a,top1b,top1s,top1c);  
    //修改半加器模块参数的模块top1,  
    input top1a,top1b;  
    output top1s,top1c;  
    defparam    U1.xor_delay=4,  
               U1.and_delay=5;  
    //名为U1的半加器实例中对参数xor_delay和参数  
    //and_delay值的修改  
    halfadder U1(top1a,top1b,top1s,top1c);  
endmodule
```

```

例3.3-7 使用defparam语句修改参数值
module halfadder(a,b,s,c);    //半加器模块
halfadder
    input a,b;
    output c,s;
        parameter xor_delay=2,and_delay=3;
    assign #xor_delay s=a^b;
    assign #and_delay c=a&b;
endmodule

module fulladder(p,q,ci,co,sum); //全加器模块
fulladder
    input p,q,ci;
    output co,sum;
    parameter or_delay=1
    wire w1,w2,w3;
    halfadder      U1(p,q,w1,w2);
    halfadder      U2(ci,w1,sum,w3);
    or #or_delay   U3(co,w2,w3);
endmodule

```

```

module
top2(top2p,top2q,top2ci,top2co,top2sum);//修改
全加器模块参数的模块top2
input top2p,top2q,top2ci;
output top2co,top2sum;
defparam  U2.U1.xor_delay=6,
        U2.U2.and_delay=7;
//名为U2的全加器实例中引用的名为U2的半加器
实例中
//对参数xor_delay和and_delay的修改
        U2.or_delay=5;
//名为U2的全加器实例中对参数or_delay值的修
改
fulladder
U2(top2p,top2q,top2ci,top2co,top2sum);
endmodule

```

门级建模

(1) Verilog HDL基本门级元件的类型

Verilog HDL内置26个基本元件，其中14个是门级元件，12个为升级元件，这26个基本元件及其分类见下表。

类型		元件
基本门	多输入门	and, nand, or, nor, xor, xnor
	多输出门	buf, not
三态门	允许定义驱动强度	buf0, buf1, not0, not1
mos 开关	无驱动强度	nmos, pmos, cmos rmos, rpmos, rcmos
双向开关	无驱动强度	tran, tranif0, tranif1
	无驱动强度	rtran, rtranif0, rtranif1
电阻	允许定义驱动强度	res, res0, res1

(2) 门级模块调用

多输入门元件调用的语法格式是：

门类型 <实例名> (<输出端口>, <输入端口1>, <输入端口2>,
, <输入端口n>);

例如：

```
and  A1 (out1, in1, in2);
```

```
or   O2 (a, b, c, d);
```

```
xor  X1(x_out, p1, p2);
```

多输出门元件调用的语法格式是：

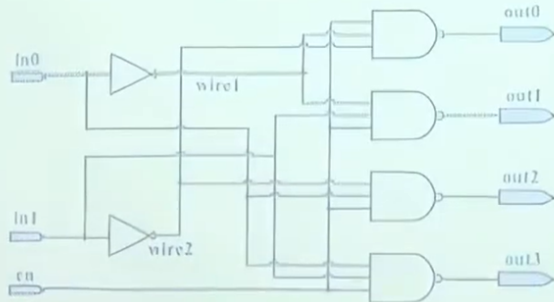
元件名 <实例名> (<输出端口1>, <输出端口2>,, <输出端口
n>, <输入端口>);

例如：

```
not  NOT_1 (out1, out2, in);
```

```
buf  BUF_1 (bufout1, bufout2, bufout3, bufin);
```

例3.3-8调用门级元件实现如图所示的2-4译码器



```
module  
decoder2_4(in0,in1,en,out0,out1,out  
2,out3);  
output out0,out1,out2,out3;  
input in0,in1,en;  
wire wire1, wire2;  
not      U1(wire1,in0),  
         U2(wire2,in1);  
nand     U3(out0,en,wire1,wire2),  
         U4(out1,en,wire1,in1),  
         U5(out2,en,in0,wire2),  
         U6(out3,en,in0,in1);  
endmodule
```

开关级建模:

Verilog HDL语言提供了十几种开关级基元，它们是实际的MOS管的抽象表示。这些开关级基元分为两大类：一类是MOS开关，一类是双向开关。每一大类又可分为电阻型（前缀用r表示）和非电阻型。本节主要以非电阻型开关为例，介绍MOS开关和双向开关。

(1) MOS开关

MOS开关模拟了实际的MOS器件的功能，包括nmos、pmos、cmos三种。

nmos和pmos的实例化语言格式是：

nmos或pmos 实例名 (out, data, control) ;

cmos开关的实例化语言格式是：

cmos 实例名 (out, data, ncontrol, pcontrol) ;

两输入与非门的CMOS电路如图所示，
其Verilog HDL程序代码是：

```
module nnand2(a,b,y);  
input a,b;  
output y;  
supply0 Gnd;  
supply1 Vdd; //supply0和supply1为  
内部参量，分别表示低电平和高电平  
wire im1;  
    pmos g1(y,Vdd,a);  
    pmos g2(y,Vdd,b);  
    nmos g3(y,im1,a);  
    nmos g4(im1,Gnd,b);  
endmodule
```

